

Elasticsearch Cluster Setup

Step-by-Step Guide: Deploying a Multi-Node Elasticsearch & Kibana Cluster with Filebeat on Debian

1. Prerequisites & System Preparation

Before beginning, ensure your Debian system satisfies the following minimum prerequisites:

- **OS:** Debian 11 (Bullseye) or Debian 12 (Bookworm).
- **Hardware Requirements (for local multi-node testing):**
 - **CPU:** 4+ vCPUs recommended.
 - **RAM:** 8 GB minimum (12 GB+ recommended to run 3 Elasticsearch nodes + Kibana smoothly).
 - **Storage:** 30 GB+ free disk space.
- **User Privileges:** root or a user with sudo access.

Step 1: Installing Docker & Docker Compose on Debian

We install the official Docker Engine package directly from Docker's repository to ensure compatibility and access to the latest stable features.

1.1 Update Package Index and Install Dependencies

Run the following command to update your local package database and install utility tools for HTTPS repository fetching and GPG key management:

```
sudo apt-get update && sudo apt-get install -y \  
ca-certificates \  
curl \  
gnupg \  
lsb-release
```

- **Why:** ca-certificates allows HTTPS connections to repositories; curl downloads GPG keys; gnupg handles cryptographic verification.

1.2 Add Docker's Official GPG Key

Download Docker's GPG key to verify package authenticity:

```
sudo install -m 0755 -d /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

- **Why:** Verifies that the Docker packages downloaded in subsequent steps are authentic and untampered.

1.3 Add Docker Repository to APT Sources

Add the repository definition to APT:

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/debian \  
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

- **Why:** Tells Debian's package manager where to locate Docker packages optimized for your CPU architecture.

1.4 Install Docker Engine and Docker Compose Plugin

Update APT indexes again and install Docker:

```
sudo apt-get update  
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

- **Why:** Installs the Docker runtime daemon (docker-ce), the command-line client (docker-ce-cli), container engine (containerd), and the Docker Compose V2 plugin (docker-compose-plugin).

1.5 Enable and Verify Docker Service

Enable Docker to start automatically upon system boot, then launch it:

```
sudo systemctl enable --now docker  
sudo docker --version  
sudo docker compose version
```

Step 2: Configuring System Requirements (Kernel Parameters)

Elasticsearch uses a mmapfs directory by default to store its indices. System defaults for maximum memory map counts are usually too low, which causes out-of-memory exceptions during startup.

2.1 Increase Virtual Memory Allocation (vm.max_map_count)

Set vm.max_map_count to at least 262144:

```
sudo sysctl -w vm.max_map_count=262144
```

- **Why:** Prevents Elasticsearch nodes from crashing with max virtual memory areas vm.max_map_count [65530] is too low errors.

2.2 Make Virtual Memory Setting Persistent

Append the setting to /etc/sysctl.conf:

```
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf  
sudo sysctl -p
```

- **Why:** Ensures the system configuration survives system reboots.

Step 3: Setting Up the Project Directory & .env Configuration

We maintain all configurations, environment variables, certificates, and Docker definitions within a dedicated project directory.

3.1 Create Directory Hierarchy

```
mkdir -p ~/elastic-cluster/{certs,filebeat}  
cd ~/elastic-cluster
```

- **Why:** certs/ holds TLS certificates shared across nodes; filebeat/ holds Filebeat configuration files.

3.2 Create .env File

Create a .env file in ~/elastic-cluster/.env containing cluster credentials, software versions, resource limits, and network ports:

```
cat << 'EOF' > .env  
# Security Passwords  
ELASTIC_PASSWORD=SuperSecretPassword123!  
KIBANA_PASSWORD=SuperSecretKibanaPassword123!  
  
# Elastic Stack Version  
STACK_VERSION=8.12.2  
  
# Cluster Port Bindings  
ES_PORT=9200  
KIBANA_PORT=5601  
  
# JVM Memory Allocation (per node)  
MEM_LIMIT=2073741824  
  
# Security / Certificate Settings  
CLUSTER_NAME=es-debian-cluster  
LICENSE=basic  
EOF
```

- **Why:** Separates credentials, resource limits, and version definitions from the container orchestration configuration, preventing hardcoded sensitive details.

Step 4: Generating TLS/SSL Certificates for Node Security

Elasticsearch 8.x mandates Transport Layer Security (TLS) for internal node-to-node communication (Transport layer) and client-to-node communication (HTTP layer).

We run a short-lived Docker container using `elasticsearch-certutil` to generate a Certificate Authority (CA) and server certificates.

4.1 Define Certificate Generation Configuration (`instances.yml`)

Create `instances.yml` to specify DNS and IP attributes for every node in our cluster:

```
cat << 'EOF' > instances.yml
instances:
  - name: es01
    dns:
      - es01
      - localhost
    ip:
      - 127.0.0.1
  - name: es02
    dns:
      - es02
      - localhost
    ip:
      - 127.0.0.1
  - name: es03
    dns:
      - es03
      - localhost
    ip:
      - 127.0.0.1
EOF
```

- **Why:** Defines explicit Subject Alternative Names (SAN) for all cluster nodes, preventing TLS certificate hostname validation failures during internal cluster discovery.

4.2 Generate Certificates Using an Ephemeral Container

Run `elasticsearch-certutil` inside a temporary container to create a Certificate Authority and generate node certificates:

```
docker run --rm \
  -v $(pwd):/usr/share/elasticsearch/config/cert_gen \
  --workdir /usr/share/elasticsearch/config/cert_gen \
  docker.elastic.co/elasticsearch/elasticsearch:8.12.2 \
  bash -c "
    bin/elasticsearch-certutil ca --silent --pem -out ca.zip && \
    unzip ca.zip && \
    bin/elasticsearch-certutil cert --silent --pem --ca-cert ca/ca.crt --ca-key ca/ca.key --in instances.yml -out certs.zip &&
```

```
\
  unzip certs.zip
"
```

- **Why:** Creates a self-signed CA (ca.crt and ca.key), issuing signed TLS key pairs (es01.crt, es01.key, etc.) for all cluster nodes without requiring a local Java/Elasticsearch installation.

4.3 Move Certificates to Output Directory & Set Permissions

Extract generated certificates to the certs/ folder and adjust file permissions:

```
mv ca es01 es02 es03 certs/
sudo chown -R 1000:1000 certs/
chmod -R 750 certs/
```

- **Why:** Elasticsearch running inside Docker defaults to UID 1000 (elasticsearch user). Setting owner permissions to 1000:1000 prevents "Permission Denied" errors when reading TLS keys.

Step 5: Drafting the Multi-Node docker-compose.yml

Create the main stack definition file docker-compose.yml containing three Elasticsearch nodes (es01, es02, es03), one Kibana instance, and shared volume definitions.

```
cat << 'EOF' > docker-compose.yml
version: "3.8"

networks:
  elastic:
    driver: bridge

volumes:
  esdata01:
    driver: local
  esdata02:
    driver: local
  esdata03:
    driver: local
  kibanadata:
    driver: local

services:
  es01:
    image: docker.elastic.co/elasticsearch/elasticsearch:${STACK_VERSION}
    container_name: es01
    labels:
      co.elastic.logs/enabled: "true"
    volumes:
      - ./certs:/usr/share/elasticsearch/config/certs
      - esdata01:/usr/share/elasticsearch/data
    ports:
      - "${ES_PORT}:9200"
```

environment:

- node.name=es01
- cluster.name=\${CLUSTER_NAME}
- cluster.initial_master_nodes=es01,es02,es03
- discovery.seed_hosts=es02,es03
- ELASTIC_PASSWORD=\${ELASTIC_PASSWORD}
- bootstrap.memory_lock=true
- xpack.security.enabled=true
- xpack.security.http.ssl.enabled=true
- xpack.security.http.ssl.key=certs/es01/es01.key
- xpack.security.http.ssl.certificate=certs/es01/es01.crt
- xpack.security.http.ssl.certificate_authorities=certs/ca/ca.crt
- xpack.security.transport.ssl.enabled=true
- xpack.security.transport.ssl.key=certs/es01/es01.key
- xpack.security.transport.ssl.certificate=certs/es01/es01.crt
- xpack.security.transport.ssl.certificate_authorities=certs/ca/ca.crt
- xpack.security.transport.ssl.verification_mode=certificate
- ES_JAVA_OPTS=-Xms1g -Xmx1g

ulimits:

- memlock:
- soft: -1
 - hard: -1

networks:

- elastic

healthcheck:

test:

```
[  
  "CMD-SHELL",  
  "curl -s -k https://localhost:9200 | grep -q 'missing authentication credentials'"  
]
```

interval: 10s

timeout: 10s

retries: 12

es02:

image: docker.elastic.co/elasticsearch/elasticsearch:\${STACK_VERSION}

container_name: es02

labels:

co.elastic.logs/enabled: "true"

volumes:

- ./certs:/usr/share/elasticsearch/config/certs
- esdata02:/usr/share/elasticsearch/data

environment:

- node.name=es02
- cluster.name=\${CLUSTER_NAME}
- cluster.initial_master_nodes=es01,es02,es03
- discovery.seed_hosts=es01,es03
- ELASTIC_PASSWORD=\${ELASTIC_PASSWORD}
- bootstrap.memory_lock=true
- xpack.security.enabled=true
- xpack.security.http.ssl.enabled=true
- xpack.security.http.ssl.key=certs/es02/es02.key

- xpack.security.http.ssl.certificate=certs/es02/es02.crt
- xpack.security.http.ssl.certificate_authorities=certs/ca/ca.crt
- xpack.security.transport.ssl.enabled=true
- xpack.security.transport.ssl.key=certs/es02/es02.key
- xpack.security.transport.ssl.certificate=certs/es02/es02.crt
- xpack.security.transport.ssl.certificate_authorities=certs/ca/ca.crt
- xpack.security.transport.ssl.verification_mode=certificate
- ES_JAVA_OPTS=-Xms1g -Xmx1g

ulimits:

memlock:

soft: -1

hard: -1

networks:

- elastic

healthcheck:

test:

```
[
  "CMD-SHELL",
  "curl -s -k https://localhost:9200 | grep -q 'missing authentication credentials'"
]
```

interval: 10s

timeout: 10s

retries: 12

es03:

image: docker.elastic.co/elasticsearch/elasticsearch:\${STACK_VERSION}

container_name: es03

labels:

co.elastic.logs/enabled: "true"

volumes:

- ./certs:/usr/share/elasticsearch/config/certs
- esdata03:/usr/share/elasticsearch/data

environment:

- node.name=es03
- cluster.name=\${CLUSTER_NAME}
- cluster.initial_master_nodes=es01,es02,es03
- discovery.seed_hosts=es01,es02
- ELASTIC_PASSWORD=\${ELASTIC_PASSWORD}
- bootstrap.memory_lock=true
- xpack.security.enabled=true
- xpack.security.http.ssl.enabled=true
- xpack.security.http.ssl.key=certs/es03/es03.key
- xpack.security.http.ssl.certificate=certs/es03/es03.crt
- xpack.security.http.ssl.certificate_authorities=certs/ca/ca.crt
- xpack.security.transport.ssl.enabled=true
- xpack.security.transport.ssl.key=certs/es03/es03.key
- xpack.security.transport.ssl.certificate=certs/es03/es03.crt
- xpack.security.transport.ssl.certificate_authorities=certs/ca/ca.crt
- xpack.security.transport.ssl.verification_mode=certificate
- ES_JAVA_OPTS=-Xms1g -Xmx1g

ulimits:

memlock:

```

    soft: -1
    hard: -1
networks:
  - elastic
healthcheck:
  test:
    [
      "CMD-SHELL",
      "curl -s -k https://localhost:9200 | grep -q 'missing authentication credentials'"
    ]
  interval: 10s
  timeout: 10s
  retries: 12

kibana:
  image: docker.elastic.co/kibana/kibana:${STACK_VERSION}
  container_name: kibana
  depends_on:
    es01:
      condition: service_healthy
    es02:
      condition: service_healthy
    es03:
      condition: service_healthy
  volumes:
    - ./certs:/usr/share/kibana/config/certs
    - kibanadata:/usr/share/kibana/data
  ports:
    - "${KIBANA_PORT}:5601"
  environment:
    - SERVERNAME=kibana
    - ELASTICSEARCH_HOSTS=https://es01:9200
    - ELASTICSEARCH_USERNAME=kibana_system
    - ELASTICSEARCH_PASSWORD=${KIBANA_PASSWORD}
    - ELASTICSEARCH_SSL_CERTIFICATEAUTHORITIES=config/certs/ca/ca.crt
  networks:
    - elastic
EOF

```

Directives Explanation

- `cluster.initial_master_nodes`: Defines master-eligible nodes involved in initial cluster bootstrapping.
- `discovery.seed_hosts`: Specifies hostname endpoints for discovery when nodes boot up.
- `bootstrap.memory_lock=true`: Locks process memory, preventing Elasticsearch memory swapping to disk.
- `ES_JAVA_OPTS=-Xms1g -Xmx1g`: Sets JVM Heap min (-Xms) and max (-Xmx) size.

Step 6: Bootstrapping and Starting the Elastic Stack

6.1 Launch Elasticsearch Nodes

Start es01, es02, and es03 in detached mode:

```
docker compose up -d es01 es02 es03
```

- **Why:** Starts the core cluster services without waiting for Kibana.

6.2 Set system Password for kibana_system

Once the cluster reaches healthy status, configure the internal built-in service account password for kibana_system:

```
docker exec -it es01 bash -c "  
bin/elasticsearch-reset-password -u kibana_system -b -p SuperSecretKibanaPassword123!  
"
```

- **Why:** Kibana uses the internal service user kibana_system to index dashboard configurations and system metadata.

6.3 Start Kibana Service

Start Kibana container after setting the password:

```
docker compose up -d kibana
```

Step 7: Verifying Cluster Health and Accessing Kibana

7.1 Check Cluster Health via REST API

Query the cluster health endpoint using curl:

```
curl --cacert certs/ca/ca.crt -u elastic:SuperSecretPassword123! https://localhost:9200/_cluster/health?pretty
```

Expected Output

```
{  
  "cluster_name" : "es-debian-cluster",  
  "status" : "green",  
  "timed_out" : false,  
  "number_of_nodes" : 3,  
  "number_of_data_nodes" : 3,  
  "active_primary_shards" : 0,  
  "active_shards" : 0,  
  "relocating_shards" : 0,  
  "initializing_shards" : 0,  
  "unassigned_shards" : 0  
}
```

7.2 Access Kibana Web Console

Open a web browser and navigate to:

http://<YOUR_DEBIAN_IP>:5601

- **Username:** elastic
- **Password:** SuperSecretPassword123!

Step 8: Configuring and Ingesting Logs with Filebeat

We now deploy Filebeat on the Debian host to forward local system logs (/var/log/auth.log and /var/log/syslog) into Elasticsearch.

8.1 Create Filebeat `cat << 'EOF' > filebeat/filebeat.yml`

```
filebeat.inputs:
- type: filestream
  id: debian-system-logs
  enabled: true
  paths:
  - /var/log/syslog
  - /var/log/auth.log

setup.kibana:
  host: "http://kibana:5601"

output.elasticsearch:
  hosts: ["https://es01:9200"]
  username: "elastic"
  password: "SuperSecretPassword123!"
  ssl.certificate_authorities: ["/usr/share/filebeat/certs/ca/ca.crt"]
  ssl.verification_mode: "full"

logging.level: info
EOF
```

File (filebeat.yml)

Create filebeat/filebeat.yml:

8.2 Set Permissions for filebeat.yml

Filebeat enforces strict security rules and refuses to start if the configuration file is writeable by non-root users:

```
sudo chown root:root filebeat/filebeat.yml
sudo chmod 600 filebeat/filebeat.yml
```

- **Why:** Prevents security violations during startup.

8.3 Append Filebeat to docker-compose.yml

Add the Filebeat service configuration block to your docker-compose.yml:

```
cat << 'EOF' >> docker-compose.yml

filebeat:
  image: docker.elastic.co/beats/filebeat:${STACK_VERSION}
  container_name: filebeat
  user: root
  networks:
    - elastic
  depends_on:
    es01:
      condition: service_healthy
    kibana:
      condition: service_started
  volumes:
    - ./filebeat/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
    - ./certs:/usr/share/filebeat/certs:ro
    - /var/log:/var/log:ro
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
    - /var/run/docker.sock:/var/run/docker.sock:ro
EOF
```

8.4 Start Filebeat and Setup Dashboards

Launch Filebeat:

```
docker compose up -d filebeat
```

Initialize Filebeat pipelines and index templates inside the container:

```
docker exec -it filebeat filebeat setup -e
```

- **Why:** Loads pre-built Filebeat dashboards, index lifecycle management (ILM) policies, and ingest pipelines into Kibana and Elasticsearch.

8.5 Verify Log Ingestion in Kibana

1. Open Kibana at `http://<YOUR_DEBIAN_IP>:5601`.
2. Navigate to **Management > Stack Management > Data Views**.
3. Create a data view with pattern `filebeat-*` and timestamp field `@timestamp`.
4. Go to **Discover** to view incoming system logs in real time.

Troubleshooting & Maintenance Tips

Issue	Cause	Solution
max virtual memory areas [65530] is too low	<code>vm.max_map_count</code> host setting is too low.	Execute <code>sudo sysctl -w vm.max_map_count=262144</code> .
Elasticsearch node exits with code 137	Out of Memory (OOM) killed by kernel.	Increase Docker memory limit or adjust <code>-Xms1g -Xmx1g</code> in <code>docker-compose.yml</code> .
Certificate validation failed	Hostname mismatch or bad CA certificate path.	Check <code>instances.yml</code> settings and verify key paths in <code>.env</code> .
Filebeat configuration file is driver-writeable	Insecure file permission on <code>filebeat.yml</code> .	Run <code>sudo chown root:root filebeat.yml && sudo chmod 600 filebeat.yml</code> .

Operational Commands Quick Reference

- **View real-time cluster logs:**
`docker compose logs -f`
- **Check running container status:**
`docker compose ps`
- **Stop the entire cluster:**
`docker compose down`
- **Stop cluster and purge all stored data volumes:**
`docker compose down -v`